



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CC5009NI Cyber Security in Computing

Assessment Weightage & Type

40% Individual Coursework 01

Year and Semester

2025 -26 Autumn Semester

Student Name: Aashutosh Poudyal

London Met ID: 24046334

College ID: NP01NT4A240026

Assignment Due Date: 19 Jan. 26

Assignment Submission Date: 18 Jan. 26

Word Count (Where Required):6805

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Final Coursework Submission

Islington College, Nepal

Document Details

Submission ID
trn:old::3618:126433939

Submission Date
Jan 18, 2026, 6:20 PM GMT+5:45

Download Date
Jan 18, 2026, 7:00 PM GMT+5:45

File Name
1.docx

File Size
262.9 KB

44 Pages

6,212 Words

29,184 Characters

20% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Small Matches (less than 8 words)

Match Groups

- 66 Not Cited or Quoted 19%**
Matches with neither in-text citation nor quotation marks
- 5 Missing Quotations 2%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 14%** Internet sources
- 3%** Publications
- 19%** Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 66 Not Cited or Quoted 19%**
Matches with neither in-text citation nor quotation marks
- 5 Missing Quotations 2%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 14% Internet sources
- 3% Publications
- 19% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	access.redhat.com	2%
2	Internet	www.coursehero.com	2%
3	Internet	doczz.net	2%
4	Submitted works	Islington College,Nepal on 2026-01-18	2%
5	Internet	www.tutorialspoint.com	<1%
6	Submitted works	South Birmingham College on 2024-06-14	<1%
7	Internet		

Table of Contents

1. Introduction to Cryptographic Systems.....	1
1.1 Security in Information system	1
1.2 CIA triad and Its Critical Role in Information Security	1
1.3 Introduction to Cryptography	3
1.5 History of Cryptography	5
1.6 Symmetric and Asymmetric Encryption Systems	8
2. Background of the Selected Cryptographic Algorithm	10
2.1 About Caesar cipher	10
2.2 Example of Caesar Cipher	11
2.2.1 Encryption Process	11
2.2.2 Decryption Process	13
2.3 Advantage and Disadvantage of Caesar Cipher.....	14
3. Development of a new Cryptographic Algorithm	15
3.1 Introduction and Research Background	15
3.2 Why Modification Was Necessary	15
3.3 Background on Mathematical and Logical Operations Used.....	17
3.3.1 Modular Arithmetic	17
3.3.2 Positional Shifting.....	17
3.3.3 Transposition (Reordering)	17
3.3.4 Key Evolution	17
3.4 Simple Hybrid Shift-Shuffle Cipher (SHSSC).....	18
3.4.2 New Methodology (SHSSC) implied	18
3.4.3 The Encryption Algorithm FOR SDSSC	22
3.4.4 The Decryption Algorithm FOR SDSSC	23
3.5 Flowchart.....	24
3.5.1 Flowchart for Encryption	24
3.5.2 Flowchart for Decryption	25

4. Test Cases (Testing) and Evaluation of the Algorithm.....	26
4.1 Testing Objectives	26
4.2 Test Case Summary Table.....	26
4.3 Testing	27
4.3.1 Test Case 1	27
4.3.2 Test Case 2	31
4.3.3 Test Case 3	36
4.3.4 Test Case 4	38
4.3.5 Test Case 5	41
5. Critical Evaluation of the New Cryptographic Algorithm	43
5.1 Strengths.....	43
5.2 Limitations.....	44
6. Conclusion	45
7. References	46
Bibliography	46
8. Appendices	47

Table of Figures

Figure 1 CIA Triad (threatscape, n.d.)	2
Figure 2 Cryptography (Gupta, 13 Oct 2022).....	3
Figure 3 Nobleman Khumhotep (TEAM, 2025).....	6
Figure 4 History of Cryptography (Sidhpurwal, n.d.)	7
Figure 5 Lucifer block on malware development (enthusiast, n.d.)	7
Figure 6 Symmetrical Encryption System (Majumder, July 18, 2025).....	9
Figure 7 Asymmetric Encryption (Majumder, July 18, 2025)	9
Figure 8 julius caesar ciphers (Pyrik, Mar 3, 2019).....	10

1. Introduction to Cryptographic Systems

1.1 Security in Information system

Security in information system is all about keeping information and system safe from people who shouldn't have access. It means putting the right policies, process, and technologies in place so only the right folks can get the data, the information stays accurate and private stuff doesn't leak. At its core security is about managing risk. You pick your defences to keep threats in check and make sure the risk doesn't get out of hand. (Holdsworth, n.d.)

Information security aims to protect systems and data against multiple threats by maintaining their confidentiality and integrity and ensuring their availability. The continuous nature of risk management requires organizations to identify assets and threats while implementing safeguards and monitoring for breaches and responding to incidents and performing recovery operations. (Holdsworth, n.d.)

1.2 CIA triad and Its Critical Role in Information Security

The CIA triad is a model for keeping information safe. It has three goals for protecting the CIA triad data: the CIA triad Confidentiality, which is keeping the CIA triad data secret; the CIA triad Integrity, which is making sure the CIA triad data is accurate; and the CIA triad Availability, which is making sure people can get to the CIA triad data when they need it. This is important for building systems that're secure and, for figuring out if a system is really secure. (Hashemi-Pour, 2023 dec 21) (GDPR, 18 June 2025)

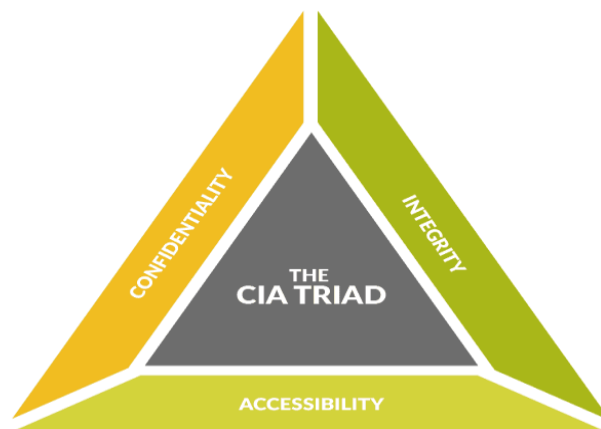


Figure 1 CIA Triad (threatscape, n.d.)

For Confidentiality: Encryption provides a method to protect data privacy. The process scrambles data so that only authorized users with the proper key can decode it. If you do not have the key you cannot read the data. The key functions as the essential component which enables the unlocking mechanism of encryption systems. Without the key the data is useless, to you. The protection of data through encryption blocks unauthorized users from obtaining access to the information. (fortinet, n.d.)

For Integrity: Integrity is about keeping your data safe from people who might try to alter it. Think of it like a label that says this data belongs to you and it has not been changed. We use things like SHA-256 to make this label. It is like a fingerprint that's unique to your data. If someone tries to change a tiny part of your data the fingerprint will be different. Then we have signatures, which are, like a lock that makes sure your data is safe and also shows who sent it. Integrity is important because it makes sure that your data is not messed with. Integrity is what keeps your data safe. (fortinet, n.d.)

For Availability: Systems do not stay online because of cryptography yet it serves as an essential component for developing secure authentication systems and access control mechanisms. The combination of digital certificates with PKI systems enables authorized personnel to access resources while preventing unauthorized attacks that

would disrupt system availability through hijacking or denial of service attacks.. (fortinet, n.d.)

1.3 Introduction to Cryptography

Cryptography generally deals with the study and practice of techniques for ensuring secure communication between two parties in the presence of a third party called adversaries. (Gupta, 5 Jun 2024)

Cryptography started out as a way to write and solve secret codes basically, a tool for hiding messages. But that old definition doesn't really fit anymore. These days, cryptography covers much more. It keeps our data safe from tampering, lets us exchange keys securely, proves who we are online, powers digital payments, makes electronic voting possible, and defends big distributed systems from attacks. (Kosinski, n.d.)

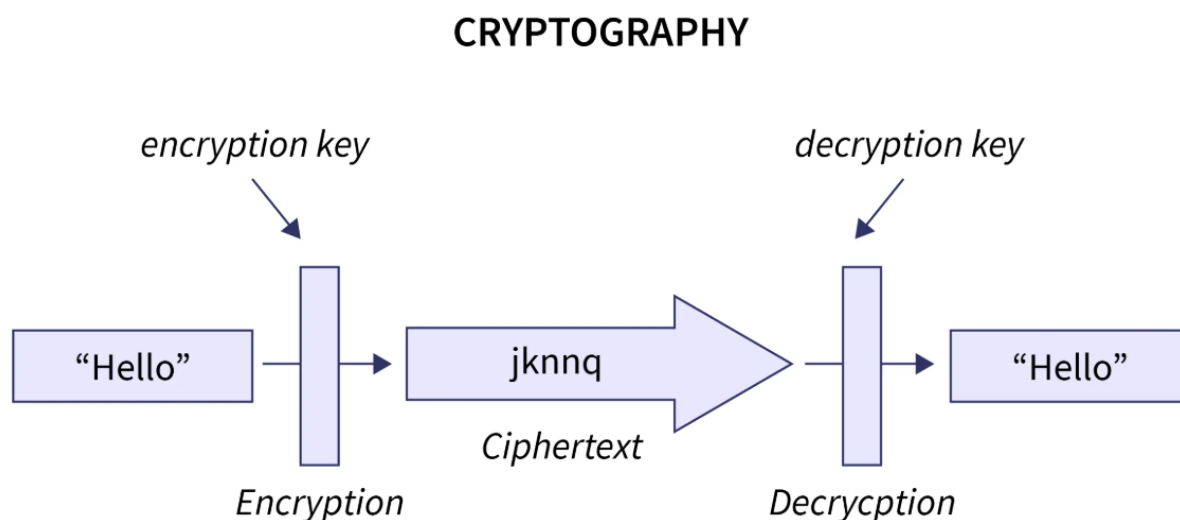


Figure 2 Cryptography (Gupta, 13 Oct 2022)

Back in the day, cryptography was more of an art people relied on clever tricks and gut feeling. Things changed in the 1970s and 80s. The field became all about science and math, with formal theories and clear definitions for what "secure" actually means. That shift reshaped computer security as we know it. (Kosinski, n.d.)

Governments and the military used to be the main players, but now cryptography is everywhere. If you shop online, log in with a password, or update your apps, you're using it. It's woven into daily digital life. At this point, cryptography isn't just a niche topic it's a core part of computer science, and it keeps systems secure for people all over the world. (Richards, mar 13 , 2024)

1.4 Key terminologies of cryptography

Plain Text

A Plain Text represents the original message which someone wants to deliver to another person. Cryptography uses the term Plain Text to refer to the message that needs to transfer to the other party.

Cipher Text

A message that remains unknown to all people or appears meaningless receives the label Cipher Text. The process of transforming an original message into an unreadable form takes place in Cryptography before the actual message is sent during communication.

Encryption

The process of converting Plain Text into Cipher Text receives the name Encryption. The encryption process serves as the essential tool in Cryptography to transmit secret messages through unsecure channels.

Encryption requires both an encryption algorithm together with a key during its implementation. An encryption algorithm establishes the specific method utilized for data encryption. The process of encryption occurs at the location where the data originates.

Decryption

The procedure of converting Cipher Text back to Plain Text represents the reverse process known as Decryption. The method of converting Cipher Text to Plain Text is known as Decryption. The decryption approach needs to be implemented by Cryptography at the receiver side to convert the unreadable message (Cipher Text) into the original message.

Key

A Key functions as a numeric or alpha numeric text or unique symbol which makes it stand out from other elements. The Key serves two purposes in encryption and decryption processes because it operates on Plain Text during encryption and Cipher Text during decryption.

The selection of key in Cryptography is essential because the security of encryption algorithm depends precisely on it.

Hashing

Hash algorithms are used to create digital fingerprints which verify that a file remains unchanged from its original state after a virus or hacker has tried to alter it. Hash functions are also generally employed by some operating framework to encrypt passwords. Hash functions provide a basic level of protection to maintain the integrity of data records.

1.5 History of Cryptography

The first known evidence of the use of cryptography was found in an inscription carved around 1900 BC, in the main chamber of the tomb of the nobleman Khnumhotep in Egypt.

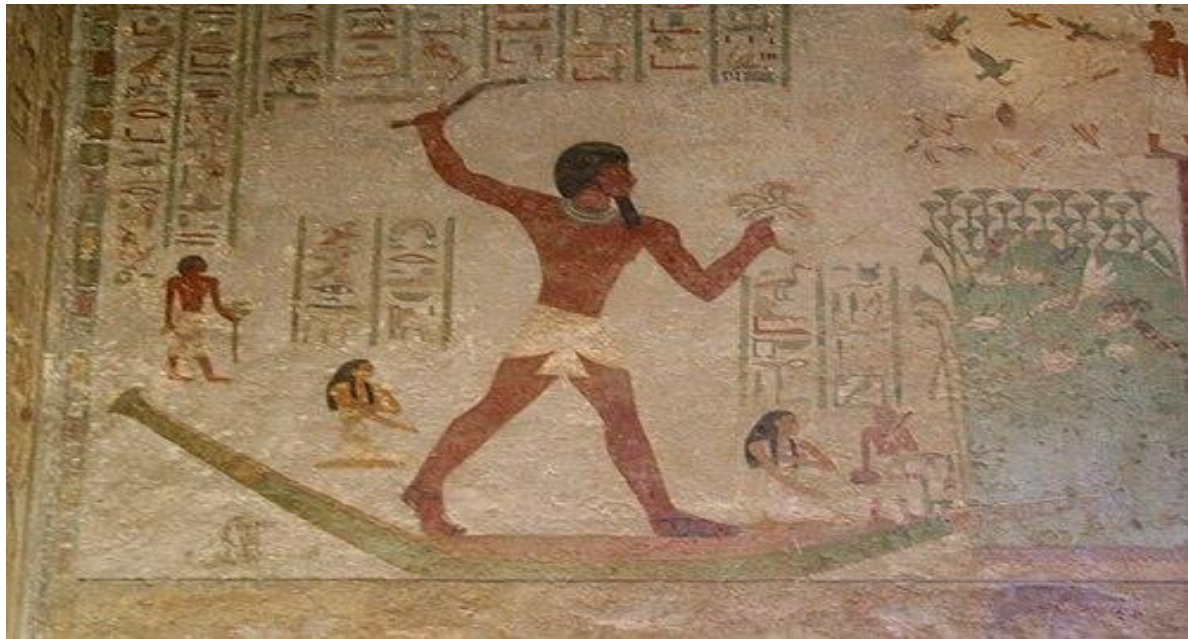


Figure 3 Nobleman Khumhotep (TEAM, 2025)

The scribe used some unusual hieroglyphic symbols here and there in place of more ordinary ones. The idea was not to conceal the message, but rather to alter it in a way that would make it seem respectable. The inscription is the first known text to involve some sort of change of the original text, even though it was not a type of secret writing. The majority of the major early civilizations have some indication of using encryption. Kautalya's classic text on statecraft, "Arthshashtra," covers India's espionage agency and includes assigning spies tasks in "secret writing" sounds like an antiquated James Bond? (Sidhpurwala, January 12, 2023)

Around 100 BC, Julius Caesar reportedly employed encryption to send confidential communications to his military leaders on the battlefield. This substitution cipher, the Caesar cipher, is frequently cited in academic discussions of historical ciphers. (An algorithm used for encryption or decryption is termed a cipher.) The substitution cipher operates by transforming every character of the original plain text into a unique character to produce the cipher text. The variant used by Caesar was a shift by 3 cipher. Each character was shifted by 3 places, so the character 'A' was replaced by 'D', 'B' was replaced by 'E', and so on. The characters would wrap around at the end, so 'X' would be replaced by 'A'. (Sidhpurwala, January 12, 2023)

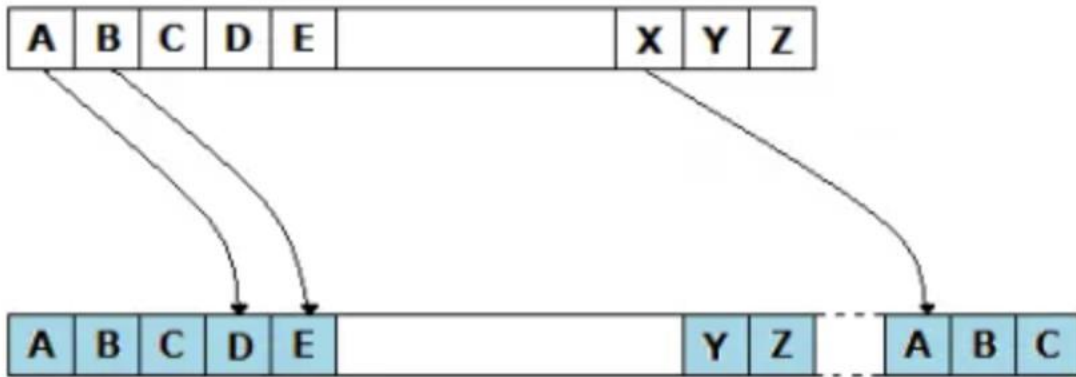


Figure 4 History of Cryptography (Sidhpurwal, n.d.)

IBM recognized the need for encryption technology during the early 1970s which led to the creation of a cryptography team under Horst-Feistel's leadership. They created a cipher which they named Lucifer. In 1973, the Nation Bureau of Standards (now called NIST) in the US put out a request for proposals for a block cipher which would become a national standard. (Sidhpurwala, January 12, 2023)

```

36 #define highsubbyte(x) ((x) >> 8)
37 #define lowsubbyte(x) ((x) & 0xFF)
38
39 void swap(char* arg1, char* arg2)
40 {
41     char tmp = *arg1;
42     *arg1 = *arg2;
43     *arg2 = tmp;
44 }
45
46 void Lucifer(char block[block_size])
47 {
48     char* lower_half = block;
49     char* upper_half = block + block_size / 2;
50
51     int key_byte_idx = decrypt ? 0 : 16;
52     const int round_count = 16;
53
54     for (int round = 0; round < round_count; round++)
55     {
56         key_byte_idx = (key_byte_idx + 1) % 16;
57
58         int transform_control_byte_idx = key_byte_idx;
59         const int step_count = 8;
60
61         for (int step = 0; step < step_count; step++)
62         {
63             char message_byte = upper_half[transform_control_byte_idx];
64             // confusion
65         }
66     }
67 }

```

```

31 c9 41 ba 45 83 56 07 ff d5 48 31 c9 41 ba f0 b5 a2 56 ff d5 4d 65 6f 77 2d 6d 65
6f 77 21 00 3d 5e 2e 2e 5e 3d 00
encrypted payload: f5 88 d6 c9 b9 32 89 53 a0 8e 9f 90 3a 4d ec cc d2 ef 7d 4b 5a 12
b7 73 86 fd dc 39 e3 4c f7 37 0f 53 c2 b9 36 5b 22 c2 0f 47 00 3d 3f c8 22 1f 15 3c
89 2f ea c7 69 83 11 c4 ae 7d 2f 71 20 be 1a 40 e5 cf 45 d8 28 79 3e 87 08 d5 03 34
35 d4 08 17 92 42 f5 90 26 bb a3 f9 88 c6 27 18 d2 c6 84 26 8e 66 70 fa cf 25 e2 60
6b 07 cb b0 0f 72 6a 1d 2e 5d 81 50 5a 7a 0c c7 ed b7 85 fe 0b c6 7f 21 9c 5a 07 1f
ba 98 89 04 84 cd 61 55 7c 8b c0 d5 73 7b 22 b7 42 3f 2a 52 1c e6 ae 07 ad 4d 74 9c
1f a0 dc ef b1 33 e7 62 12 04 a9 ec 4b 43 41 6f bd 3b 5f f0 ce 0e c3 06 dc c3 34 b7
63 05 82 6b 1f 69 8d 32 fd 6a 97 f4 c4 be d8 be 2f 35 d4 45 5d 2f 7a d6 78 aa 9e a6
81 c6 83 18 40 e3 8a 96 a2 f5 9b 2f c4 90 79 c9 3d 62 4a ac 7a 64 b9 7a 9f f8 07 45
75 a7 8c 53 3e 98 9f 6a e2 08 92 ae 95 4a 5a 54 8d 4c 80 17 89 45 66 d2 63 b3 6b
6c 72 a4 01 8c c1 14 c5 e9 6a 9f 69 9a
decrypted payload: fc 48 81 e4 f0 ff ff 00 00 00 41 51 41 50 52 51 56 48 31
d2 65 48 8b 52 60 3e 48 8b 52 18 3e 48 3e 48 8b 72 50 3e 48 0f b7 4a 4a 4d
31 c9 48 31 c0 ac 3c 61 7c 02 2c 20 41 c1 c9 0d 41 01 c1 e2 ed 52 41 51 3e 48 8b 52
20 3e 8b 42 3c 48 01 d0 3e 8b 80 88 00 00 00 48 85 c0 74 6f 48 01 d0 50 3e 8b 48 18
3e 44 8b 40 20 49 01 d0 e3 5c 48 ff c9 3e 41 8b 34 88 48 01 d6 4d 31 c9 48 31 c0 ac
41 c1 c9 0d 41 01 c1 38 e0 75 f1 3e 4c 03 4c 24 08 45 39 d1 75 d6 58 3e 44 8b 40 24
49 01 d0 66 3e 41 8b 0c 48 3e 44 8b 40 1c 49 01 d0 3e 41 8b 04 88 48 01 d0 41 58 41
58 5e 59 5a 41 58 41 59 41 5a 48 83 ec 20 41 52 ff e0 58 41 59 5a 3e 48 8b 12 e9 49
ff ff ff 5d 49 c7 c1 00 00 00 00 3e 48 8d 95 fe 00 00 00 3e 4c 8d 85 09 01 00 00 48
31 c9 41 ba 45 83 56 07 ff d5 48 31 c9 41 ba f0 b5 a2 56 ff d5 4d 65 6f 77 2d 6d 65
6f 77 21 00 3d 5e 2e 2e 5e 3d 00

```

Figure 5 Lucifer block on malware development (enthusiast, n.d.)

The buyers recognized that they purchased numerous commercial products which lacked proper cryptographic protection. The Data Encryption Standard (DES) became the accepted name for Lucifer after its official approval. The following years after 1997 saw DES become vulnerable to an exhaustive search attack which successfully broke the system. The main problem with DES was the small size of the encryption key. The rising computing power allowed attackers to try every potential key combination which would reveal the complete original message.

Lucifer ultimately gained acceptance and was renamed the Data Encryption Standard, or DES. By 1997 and in the years that followed, DES had been compromised through a brute force attack. The key issue with DES was the limited size of its encryption key. As computational power grew, it became feasible to systematically attempt all possible key combinations to retrieve a conceivable plaintext message. (Schneider, n.d.)

NIST released another call for proposals for a new block cipher in 1997.

Fifty submissions were received. It adopted Rijndael in 2000 and dubbed it the Advanced Encryption Standard, or AES. AES is now a commonly used standard for symmetric encryption. (Sidhpurwala, January 12, 2023)

1.6 Symmetric and Asymmetric Encryption Systems

Symmetric encryption operates with a single key which serves the purpose of both encryption and decryption for data protection. The sender and receiver maintain a shared secret key which protects all data transmissions. A trusted party generates a secret key for Symmetric encryption which serves as the core component for all cryptographic operations making it impossible to compromise the key. The sender converts the original data into ciphertext by using the key to encrypt the data into unreadable code. The message remains protected during network transmission because unauthorized parties require the key to access it. Cracking it by guessing would take so much computing power, it's basically impossible. The encrypted message travels through the network without any additional security measures. The data will reach its destination securely as long as no third parties interfere with it or cause any data loss.. (Majumder, July 18, 2025)

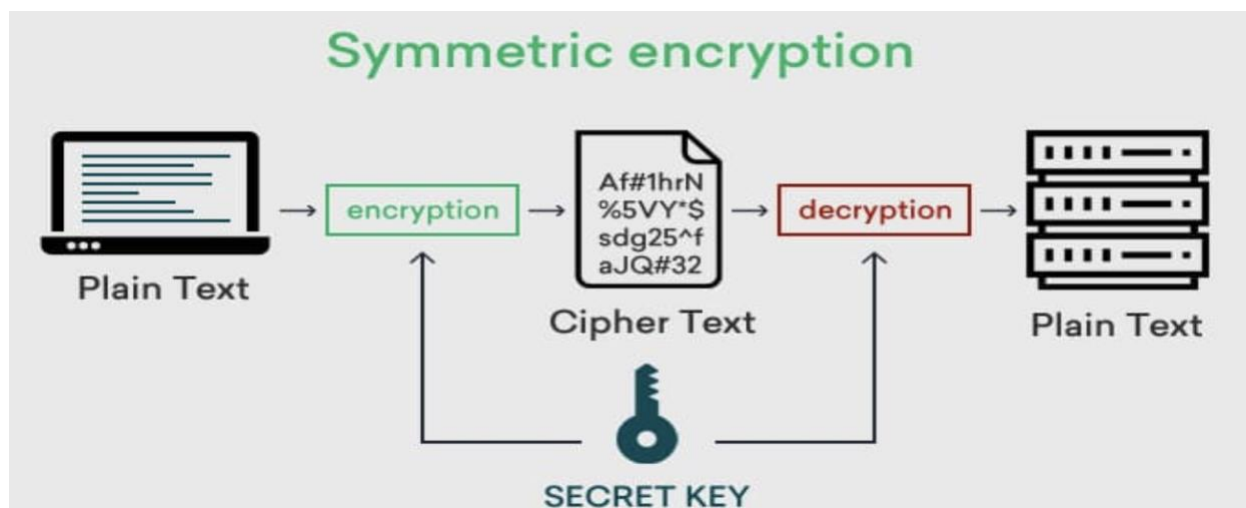


Figure 6 Symmetrical Encryption System (Majumder, July 18, 2025)

Asymmetric encryption uses two separate keys which serve different purposes for data encryption and decryption. Both keys can be used for encryption, but the encrypted data cannot be decrypted by the same key. After generating the keys, the sender grabs the public key and uses it to lock up the data. The public key method does not allow you to decrypt the ciphertext which makes it impossible to use the same key for this purpose. The sender sends this encrypted message through the network which marks the end of the process. No extra layers of security. The public key remains accessible to everyone yet no one else has the ability to decrypt the message. (Majumder, July 18, 2025)

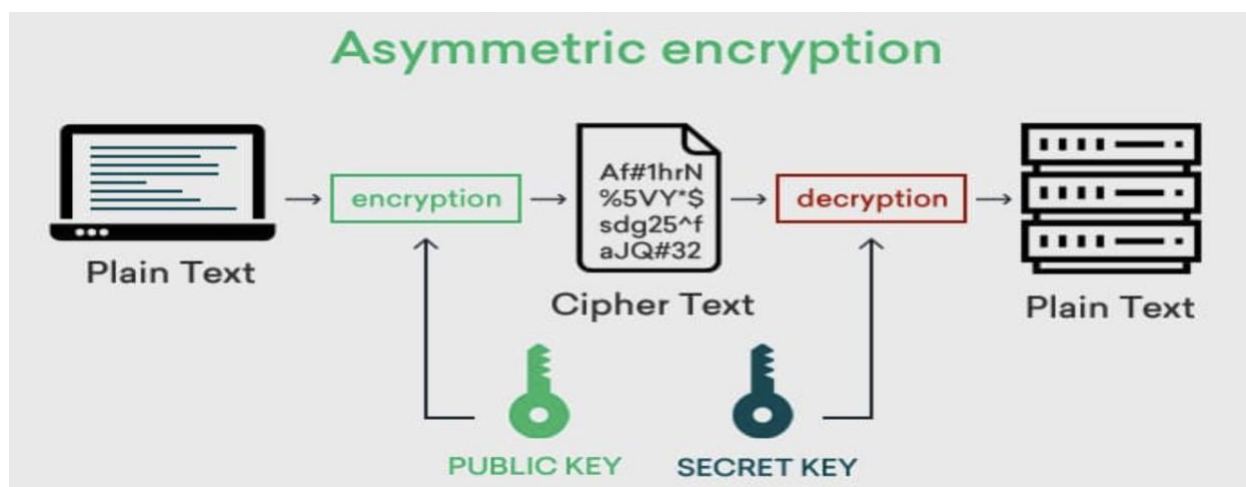


Figure 7 Asymmetric Encryption (Majumder, July 18, 2025)

2. Background of the Selected Cryptographic Algorithm

2.1 About Caesar cipher

The Caesar Cipher is one of the earliest known ciphers. Julius Caesar came up with it around 100 BC to send secret messages to his generals. If one of his messages got intercepted, his opponent could not read them. This obviously gave him a great strategic advantage. (Wickramasinghe, n.d.)



Figure 8 julius caesar ciphers (Pyrik, Mar 3, 2019)

This technique takes each letter in the alphabet and moves it over by a set number. So, if you shift everything by three, 'A' turns into 'D,' 'B' turns into 'E,' and you just keep going like that. Simple, right? But don't let that fool you the Caesar Cipher actually laid the foundation for the cryptography we use today. In this article, I'll Walk you through how the Caesar Cipher works, why it matters, and what it brought to the world of

cryptography, including what it gets right and where it falls short. (geeksforgeeks, 23 Jul, 2025)

The ROT13 cipher operates as an updated version of the Caesar cipher which was developed after his initial work. The ROT13 cipher operates through the same letter-shifting method which the Caesar cipher uses by moving every letter 13 positions ahead in the alphabet. The process of shifting 13 positions becomes convenient because ROT13 encryption functions as its own decryption method which follows the initial encryption stage. Users can access basic ROT13 encryption tools through the standard Linux and UNIX operating systems which include these utilities in their default installation package. Simple systems that use basic transposition methods exist in multiple instances. (Wickramasinghe, n.d.) (geeksforgeeks, 23 Jul, 2025)

2.2 Example of Caesar Cipher

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
13	14	15	16	17	18	19	20	21	22	23	24	25

This TABLE is a kind of reference table, from which we encrypt and decrypt values using the Caesar cipher.

Assumptions

- Plaintext: MY NAME IS AASHU
- Key shift (K) = 5

2.2.1 Encryption Process

Formula for encryption: $E = (P + K) \text{ MOD } 26$

Where:

- **E** = encrypted value
- **P** = plaintext value
- **K** = key shift

Step-by-step encryption:

$$\mathbf{M} = (12 + 5) \bmod 26 = 17 = \mathbf{R}$$

$$\mathbf{Y} = (24 + 5) \bmod 26 = 3 = \mathbf{D}$$

$$\mathbf{N} = (13 + 5) \bmod 26 = 18 = \mathbf{S}$$

$$\mathbf{A} = (0 + 5) \bmod 26 = 5 = \mathbf{F}$$

$$\mathbf{M} = (12 + 5) \bmod 26 = 17 = \mathbf{R}$$

$$\mathbf{E} = (4 + 5) \bmod 26 = 9 = \mathbf{J}$$

$$\mathbf{I} = (8 + 5) \bmod 26 = 13 = \mathbf{N}$$

$$\mathbf{S} = (18 + 5) \bmod 26 = 23 = \mathbf{X}$$

$$\mathbf{A} = (0 + 5) \bmod 26 = 5 = \mathbf{F}$$

$$\mathbf{A} = (0 + 5) \bmod 26 = 5 = \mathbf{F}$$

$$\mathbf{S} = (18 + 5) \bmod 26 = 23 = \mathbf{X}$$

$$\mathbf{H} = (7 + 5) \bmod 26 = 12 = \mathbf{M}$$

$$\mathbf{U} = (20 + 5) \bmod 26 = 25 = \mathbf{Z}$$

Ciphertext Result :

Plaintext: MY NAME IS AASHU

Ciphertext: RD SFRJ NX FFXMZ

2.2.2 Decryption Process

Formula for decryption: $D = (C - K) \bmod 26$

Where:

- **D** = decrypted value (original plaintext)
- **C** = ciphertext value
- **K** = key shift

Step-by-step Decryption Process:

$$R = (17 - 5) \bmod 26 = 12 = \mathbf{M}$$

$$D = (3 - 5) \bmod 26 = 24 = \mathbf{Y}$$

$$S = (18 - 5) \bmod 26 = 13 = \mathbf{N}$$

$$F = (5 - 5) \bmod 26 = 0 = \mathbf{A}$$

$$R = (17 - 5) \bmod 26 = 12 = \mathbf{M}$$

$$J = (9 - 5) \bmod 26 = 4 = \mathbf{E}$$

$$N = (13 - 5) \bmod 26 = 8 = \mathbf{I}$$

$$X = (23 - 5) \bmod 26 = 18 = \mathbf{S}$$

$$F = (5 - 5) \bmod 26 = 0 = \mathbf{A}$$

$$F = (5 - 5) \bmod 26 = 0 = \mathbf{A}$$

$$X = (23 - 5) \bmod 26 = 18 = \mathbf{S}$$

$$M = (12 - 5) \bmod 26 = 7 = \mathbf{H}$$

$$Z = (25 - 5) \bmod 26 = 20 = \mathbf{U}$$

Decryption Result:

Here we can see that the plain text of the ciphertext “RD SFRJ NX FFXMZ” is “MY NAME IS AASHU”.

2.3 Advantage and Disadvantage of Caesar Cipher

Pros	Cons
• The system provides a simple process for its creation and operation and user interface. • You can improve security by adding multiple shifts together with keyword-based modifications to the existing system. • Requires minimal pre-shared information. • It is great for beginners to learn encryption. (Wickramasinghe, n.d.)	• Not secure against modern decryption techniques. • The system operates with a restricted set of possible keys which makes it vulnerable to brute force attacks because of its small keyspace. • Not suitable for encrypting long texts due to ease of cracking. • The system fails to provide security for communication because attackers can easily decipher (Wickramasinghe, n.d.)

3. Development of a new Cryptographic Algorithm

3.1 Introduction and Research Background

A single fixed number shift defines the Caesar cipher as a monoalphabetic substitution cipher. The shift of 3 moves the letter 'A' to 'D' and 'B' to 'E' and continues this pattern for the rest of the alphabet. The system operates through basic principles which allow attackers to break the cipher by running all 26 possible shift combinations or by analyzing letter frequency patterns. The Vigenère cipher employs a keyword system which generates diverse shifts to enhance its security. The Vigenère cipher provides better security than Caesar but it contains patterns which attackers can detect through repeated-key analysis and frequency attack techniques.

The developed hybrid methods which combine various approaches to solve these restrictions. The common practice involves using substitution methods to shift letters together with transposition techniques which rearrange the order of letters. The objective requires introducing mathematical complexity to block predictable patterns from appearing in the ciphertext.

3.2 Why Modification Was Necessary

Classical ciphers have two main weaknesses:

1. Predictable patterns:

Simple substitution methods like the Caesar cipher operate through a single uniform letter shift across all characters in the alphabet. The consistent structure of the cipher allows attackers to break it through frequency analysis and pattern

recognition techniques. The Vigenère cipher becomes vulnerable when repeated keywords show identifiable patterns.

2. Key limitations:

The encryption system becomes vulnerable because of fixed or repeated keys which make it possible for attackers to predict the keys. The lack of key variation allows attackers to use known key segments for message decryption.

The new algorithm solves these weaknesses through the following solutions.

- Dynamic shifts :

The encryption process uses numeric keys and character positions to create dynamic shifts which produce different encrypted results for identical letters.

- Transposition :

Transposition works by shifting characters around to create new patterns which strengthens the diffusion process.

- Key evolution :

The process of key evolution requires a new key to be generated after each round because it prevents attackers from predicting future keys.

The modifications improve both substitution complexity and plaintext influence spread which are basic encryption principles from Claude Shannon. The modifications improve the algorithm's security level which makes it more difficult for modern cryptanalysis to break.

3.3 Background on Mathematical and Logical Operations Used

The new algorithm requires knowledge of its underlying mathematical and logical principles before introduction.

3.3.1 Modular Arithmetic

The fundamental mathematical operations of modular arithmetic serve as the base structure for cryptography. In encryption systems alphabetic text operations use modulo 26 because English alphabets contain 26 letters. The wrap-around operations of modular arithmetic produce results between 0 and 25, which correspond to the letters A, through Z.

3.3.2 Positional Shifting

The system uses variable shifts which depend on the following factors instead of fixed shifts:

- The key digits
- The position of the character in the message

3.3.3 Transposition (Reordering)

The operation of transposition rearranges characters through logical operations without changing their original values. The process of reversing a string serves as a basic example of transposition.

The diffusion process of transposition creates position mixing, which makes it difficult to trace the relationship between plaintext and ciphertext.

3.3.4 Key Evolution

The encryption key undergoes changes throughout the encryption process according to the key evolution method. The system operates with a structure which resembles

present-day block ciphers through its use of "round keys" that stem from an initial key to enhance protection against basic attacks.

3.4 Simple Hybrid Shift-Shuffle Cipher (SHSSC)

3.4.1 Introduction to the Simple Hybrid Shift-Shuffle Cipher (SHSSC) and Need for Modification

The Simple Hybrid Shift-Shuffle Cipher (SHSSC) is a newly designed symmetric encryption algorithm developed to overcome the limitations of classical cryptographic techniques such as the Caesar Cipher and Vigenère Cipher. Traditional substitution-based encryption systems use static or recurring shift patterns which expose them to modern cryptanalysis methods that include frequency analysis and brute-force attacks. The Vigenère cipher along with other polyalphabetic methods becomes vulnerable to attacks when keys are short or repeated because this allows attackers to find patterns in the ciphertext.

To address these weaknesses, SHSSC introduces a hybrid encryption approach that combines multiple shift operations, character position shuffling, and key-driven iterative processing. The encryption process of SHSSC uses a numeric key which breaks down into individual digits to perform multiple rounds of encryption instead of using a single fixed shift. Each round begins with a character shift based on the current key digit followed by a shuffle operation that disrupts the regular patterns in the ciphertext.

3.4.2 New Methodology (SHSSC) implied

The Simple Hybrid Shift-Shuffle Cipher (SHSSC) follows a hybrid encryption methodology that combines substitution and transposition techniques to enhance security while maintaining simplicity. The algorithm operates in multiple rounds, each controlled by a digit of a numeric key.

In each round, plaintext characters are first shifted according to the current key digit and their position within the text, introducing variability and reducing predictable patterns.

The characters undergo a reversal operation which scrambles their positions to generate enhanced diffusion and hide statistical patterns. The process continues until all key digits have been processed which creates multiple encryption layers to defend against standard cryptanalysis attacks. Decryption applies the same methodology in reverse order, ensuring that the original plaintext is accurately recovered using the same key.

1 Encryption Process

Given:

- **Plaintext:** AASHU
- **Numeric Key:** 314
- **Alphabet Mapping:** A = 0, B = 1, ..., Z = 25

Step 1: Write the Plaintext

The original message to be encrypted is written in uppercase letters.

Plaintext: AASHU

Step 2: Convert Plaintext Letters to Numbers

Each letter is converted into its numerical equivalent.

Letter	A	A	S	H	U
Value	0	0	18	7	20

Step 3: Apply Key Digits One by One

The numeric key 314 is applied digit by digit, where each digit represents one encryption round.

Round 1: Shift = 3**Step 1: Add 3 to Each Value**

$$0 + 3 = 3 \text{ (D)}$$

$$0 + 3 = 3 \text{ (D)}$$

$$18 + 3 = 21 \text{ (V)}$$

$$7 + 3 = 10 \text{ (K)}$$

$$20 + 3 = 23 \text{ (X)}$$

Result after shifting: DDVKX**Step 2: Shuffle Letters**DDVKX $\rightarrow$ XKVDD**Round 2: Shift = 1****Step 1: Shift Each Letter by 1**

$$X \rightarrow Y$$

$$K \rightarrow L$$

$$V \rightarrow W$$

$$D \rightarrow E$$

$$D \rightarrow E$$

Result after shifting: YLWEE**Step 2: Shuffle Letter**YLWEE $\rightarrow$ EEWLY**Round 3: Shift = 4****Step 1: Shift Each Letter by 4**

$$E \rightarrow I$$

$$E \rightarrow I$$

$$W \rightarrow A$$

$$L \rightarrow P$$

$$Y \rightarrow C$$

Result after shifting: IIAPC**Step 2: Shuffle Letter**

IIAPC $\rightarrow$ CPAII

Final Encrypted Output

Ciphertext: CPAII

2 Decryption Process

Given:

- **Ciphertext:** CPAII
- **Numeric Key:** 314

Step 1: Reverse Shuffle

Undo the last shuffle by reversing the ciphertext.

CPAII $\rightarrow$ IIAPC

Step 2: Reverse Shift Using Last Key Digit (4)

Subtract 4 from each letter.

I - 4 = E

I - 4 = E

A - 4 = W

P - 4 = L

C - 4 = Y

Result:EEWLY

Step 3: Continue for Remaining Key Digits

Reverse Shuffle

EEWLY $\rightarrow$ YLWEE

Reverse Shift (Key digit = 1)

Y - 1 = X

L - 1 = K

W - 1 = V

$E - 1 = D$

$E - 1 = D$

Result:XKVDD

Reverse Shuffle

XKVDD $\rightarrow$ DDVKX

Reverse Shift (Key digit = 3)

$D - 3 = A$

$D - 3 = A$

$V - 3 = S$

$K - 3 = H$

$X - 3 = U$

Final Decrypted Output

Recovered Plaintext:AASHU

3.4.3 The Encryption Algorithm FOR SDSSC

Step 1: Write down the plaintext you want to encrypt using uppercase letters only.

Step 2: Choose a numeric key (for example, 314).

Step 3: Assign numerical values to each letter of the plaintext using the rule:

$A = 0, B = 1, C = 2, \dots, Z = 25.$

Step 4: Take the first digit of the key.

Step 5: Add this digit to the numerical value of each plaintext letter.

- If the result is greater than 25, subtract 26 (modulo 26).

Step 6: Convert the resulting numbers back into letters.

Step 7: Reverse (shuffle) the entire string of letters.

Step 8: Repeat Steps 4 to 7 for each remaining digit of the key, processing the digits from left to right.

Step 9: After all key digits have been applied, the final reversed string is the ciphertext.

3.4.4 The Decryption Algorithm FOR SDSSC

Step 1: Write down the ciphertext you want to decrypt.

Step 2: Use the same numeric key that was used for encryption.

Step 3: Start with the last digit of the key.

Step 4: Reverse (shuffle back) the entire ciphertext string.

Step 5: Assign numerical values to each letter using:

A = 0, B = 1, ..., Z = 25.

Step 6: Subtract the key digit from each numerical value.

- If the result is negative, add 26 (modulo 26).

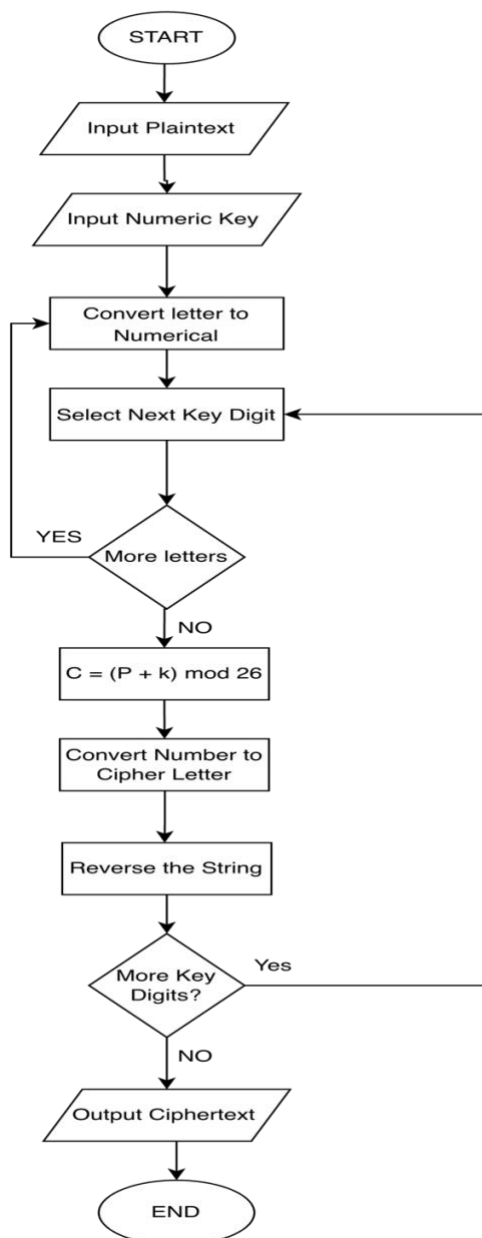
Step 7: Convert the resulting numbers back into letters.

Step 8: Repeat Steps 3 to 7 for the remaining key digits, moving backwards through the key.

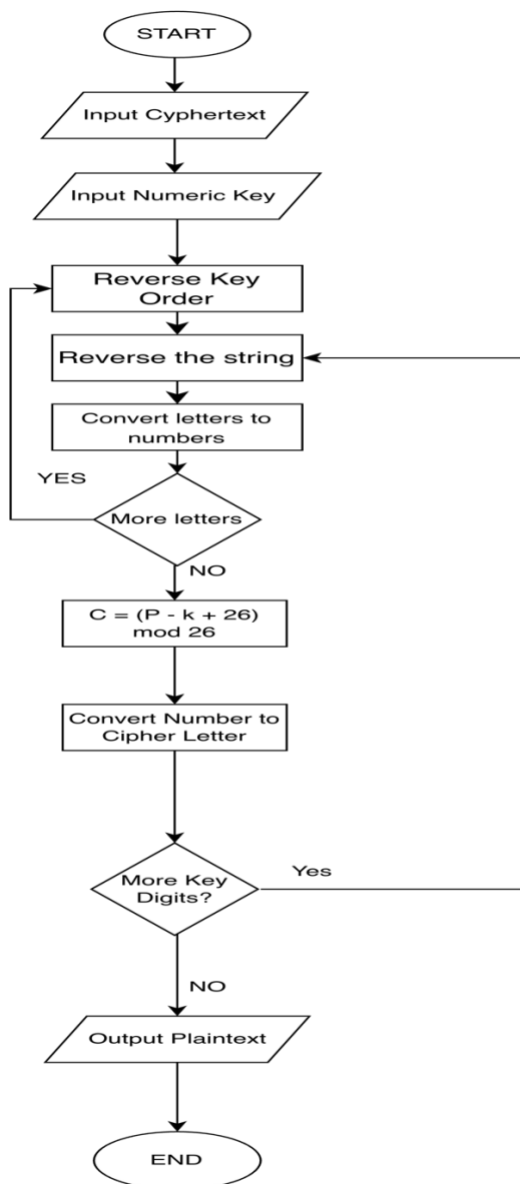
Step 9: After all key digits have been processed, the final result is the recovered plaintext.

3.5 Flowchart

3.5.1 Flowchart for Encryption



3.5.2 Flowchart for Decryption



4. Test Cases (Testing) and Evaluation of the Algorithm

4.1 Testing Objectives

The objectives of the testing phase are as follows:

- To verify that encryption and decryption work correctly using the same numeric key
- To ensure complete reversibility of the algorithm
- To examine the effect of different keys on the same plaintext
- To evaluate resistance against frequency and pattern based attacks
- To test the algorithm with varying plaintext lengths

4.2 Test Case Summary Table

Test Case ID	Test Scenario	Plaintext Type	Key Type	Purpose
TC-01	Basic encryption & decryption	Short text	Numeric key	Validate correctness
TC-02	Repeated character input	Repetitive characters	Numeric key	Test pattern resistance
TC-03	Same plaintext with different keys	Short text	Multiple keys	Test key dependency
TC-04	Long plaintext encryption	Long text	Numeric key	Test scalability
TC-05	Uppercase and Lowercase Handling	Short text	Numeric key	Evaluate handling of uppercase and lowercase characters

4.3 Testing

4.3.1 Test Case 1

Basic Encryption and Decryption

Encryption Process

Plaintext = AASHU

Numeric = Key 314

Alphabet Mapping = A = 0, B = 1, ..., Z = 25

Step 1: Plaintext Input

AASHU

Step 2: Convert Plaintext to Numerical Values

Character	A	A	S	H	U
Value	0	0	18	7	20

Step 3: Apply Numeric Key Digit by Digit

The numeric key is 314

Round 1 – Key Digit = 3

Original value	Calculation	New Value	Encrypted Letter
0	$(0+3) \bmod 26$	3	D
0	$(0+3) \bmod 26$	3	D
18	$(18+3) \bmod 26$	21	V
7	$(7+3) \bmod 26$	10	K
20	$(20+3) \bmod 26$	23	X

Output after shifting: DDVKX

Step 3.2: Shuffle (Reverse) Operation

DDVKX → XKVDD

Round 2 – Key Digit = 1

Character	X	K	V	D	D
Value	23	10	21	3	3
Shifted	24	11	22	4	4
Letter	Y	L	W	E	E

Output after shifting: YLWEE

Step 3.3: Shuffle (Reverse) Operation

YLWEE → EEWLY

Round 3 – Key Digit = 4

Character	E	E	W	L	Y
Value	4	4	22	11	24
Shifted	8	8	0	15	2
Letter	I	I	A	P	C

Output after shifting: IIAPC

Step 3.4: Shuffle (Reverse) Operation

IIAPC → CPAII

Final Encryption Result

Plaintext = AASHU

Ciphertext = CPAII

Decryption Process

Step 1: Ciphertext Input

CPAIL

Step 2: Apply Key in Reverse Order

Decryption applies key digits in reverse order: $4 \rightarrow 1 \rightarrow 3$

Round 1 – Key Digit = 4

Step 2.1: Reverse Shuffle

CPAIL $\rightarrow$ IIAPC

Step 2.2: Reverse Shift (-4)

Character	I	I	A	P	C
Value	8	8	0	15	2
Shifted(-4)	4	4	22	11	24
Letter	E	E	W	L	Y

Output: EEWLY

Round 2 – Key Digit = 1

Step 2.3: Reverse Shuffle

EEWLY $\rightarrow$ YLWEE

Step 2.4: Reverse Shift (-1)

Character	Y	L	W	E	E
Value	24	11	22	4	4
Shifted(-1)	23	10	21	3	3
Letter	X	K	V	D	D

Output: XKVDD

Round 3 – Key Digit = 3

Step 2.5: Reverse Shuffle

XKVDD → DDVKX

Step 2.6: Reverse Shift (-3)

Character	D	D	V	K	X
Value	3	3	21	10	23
Shifted(-3)	0	0	18	7	20
Letter	A	A	S	H	U

Final Decryption Result

Ciphertext = CPAII

Recovered Plaintext = AASHU

4.3.2 Test Case 2

Repeated Character Pattern Test

Encryption Process

Plaintext AAAAA

Numeric Key 246

Alphabet Mapping A = 0, B = 1, ..., Z = 25

Step 1: Plaintext Input

AAAAA

Step 2: Convert Plaintext to Numerical Values

Character	A	A	A	A	A
Value	0	0	0	0	0

Step 3: Apply Numeric Key Digit by Digit

The numeric key is **246**

Round 1 – Key Digit = 2

Step 3.1: Shift Operation

Original Value	Calculation	New Value	Encrypted Letter
0	$(0+2) \text{ mode } 26$	2	C
0	$(0+2) \text{ mode } 26$	2	C
0	$(0+2) \text{ mode } 26$	2	C
0	$(0+2) \text{ mode } 26$	2	C
0	$(0+2) \text{ mode } 26$	2	C

Output after shifting:CCCCC

Step 3.2: Shuffle (Reverse) Operation

CCCCC → CCCCC

Round 2 – Key Digit = 4

Step 3.3: Shift Operation

Character	C	C	C	C	C
value	2	2	2	2	2
Shifted (+4)	6	6	6	6	6
letter	G	G	G	G	G

Output after shifting:GGGGG

Step 3.4: Shuffle (Reverse) Operation

GGGGG → GGGGG

Round 3 – Key Digit = 6

Step 3.5: Shift Operation

Character	G	G	G	G	G
value	6	6	6	6	6
Shifted (+4)	12	12	12	12	12
letter	M	M	M	M	M

Output after shifting:MMMMM

Step 3.6: Shuffle (Reverse) Operation

MMMMM → MMMMM

Final Encryption Result

Plaintext = AAAAA

Ciphertext = MMMMM

Decryption Process

Step 1: Ciphertext Input

MMMMM

Step 2: Apply Key in Reverse Order

Decryption key order: 6 → 4 → 2

Round 1 – Key Digit = 6

Step 2.1: Reverse Shuffle

MMMMM → MMMMM

Step 2.2: Reverse Shift (-6)

Character	M	M	M	M	M
Value	12	12	12	12	12
Shifted(-6)	6	6	6	6	6
Letter	G	G	G	G	G

Output:GGGG

Round 2 – Key Digit = 4

Step 2.3: Reverse Shuffle

GGGGG → GGGGG

Step 2.4: Reverse Shift (-4)

Character	G	G	G	G	G
Value	6	6	6	6	6
Shifted(-6)	2	2	2	2	2
Letter	C	C	C	C	C

Output:CCCCC

Round 3 – Key Digit = 2

Step 2.5: Reverse Shuffle

CCCCC → CCCCC

Step 2.6: Reverse Shift (-2)

Character	C	C	C	C	C
Value	2	2	2	2	2
Shifted(-6)	0	0	0	0	0
Letter	A	A	A	A	A

Final Decryption Result

Ciphertext= MMMMM

Recovered Plaintext = AAAAA

4.3.3 Test Case 3

Same Plaintext with Different Keys

Encryption Comparison

Plaintext AASHU

Keys Used 314 and 527

Alphabet Mapping A = 0 B = 1 ... Z = 25

Step 1: Plaintext Numerical Representation

Character	A	A	S	H	U
Value	0	0	18	7	20

Step 2: Encryption Comparison

Encryption Using Key 314

Round	Key Digit	After Shift	After Shuffle
Round 1	3	DDVKX	XKVDD
Round 2	1	YlwEE	EEWLY
Round 3	4	IIAPC	CPAII

Final Ciphertext (Key 314): CPAII

Encryption Using Key 527

Round	Key Digit	After Shift	After Shuffle
Round 1	5	FFXMZ	ZMXFF
Round 2	2	BOZHH	HHZOB
Round 3	7	OOGVI	IVGOO

Final Ciphertext (Key 527): IVGOO

Step 3: Ciphertext Comparison

Plaintext	Key Used	Ciphertext Output
AASHU	314	CPAIL
AASHU	527	IVGOO

Step 4: Decryption Validation

Ciphertext	Key Used	Recovered Plaintext
CPAIL	314	AASHU
IVGOO	527	AASHU

The SHSSC algorithm generates different results from its key when using this alternative testing method which confirms its key-dependent nature. The plaintext text stays the same but different numeric keys will create multiple versions of ciphertext output. The security level of this property exceeds that of traditional ciphers because the Caesar cipher generates predictable results from its restricted number of keys.

4.3.4 Test Case 4

Long Plaintext Encryption and Decryption

Encryption Process

Plaintext = CYBERSECURITY

Key Used = 428

Alphabet Mapping = A = 0 ... Z = 25

Step 1: Plaintext Numerical Representation

Character	C	Y	B	E	R	S	E	C	U	R	I	T	Y
Value	2	24	1	4	17	18	4	2	20	17	8	19	24

Step 2: Encryption Process

Round 1 – Key Digit: 4

- Shift each character by +4
- Apply shuffle operation

Stage	Output
After Shift	G C F I V W I G Y V M X C
After Shuffle	C X V Y I G V M F I G W C

Round 2 – Key Digit: 2

Stage	Output
After Shift	E Z X A K I X O H K I Y E
After Shuffle	Y I K X H E Z O A I K X E

Round 3 – Key Digit: 8

Stage	Output
After Shift	G Q S F P M H W I Q S F M
After Shuffle	M F S Q I W H P F S Q G M

Final Ciphertext Output : MFSQIWH PFSQGM

Decryption Process

Round 1 – Reverse of Encryption Round 3 (Key Digit: 8)

Stage	Output
Before Reverse shuffle	M F S Q I W H P F S Q G M
After Reverse shuffle	G Q S F P M H W I Q S F M
After Reverse shuffle (-2)	Y I K X H E Z O A I K X E

Round 2 – Reverse of Encryption Round 2 (Key Digit: 2)

Stage	Output
Before Reverse shuffle	Y I K X H E Z O A I K X E
After Reverse shuffle	E Z X A K I X O H K I Y E
After Reverse shuffle (-2)	C X V Y I G V M F I G W C

Round 3 – Reverse of Encryption Round 1 (Key Digit: 4)

Stage	Output
Before Reverse shuffle	C X V Y I G V M F I G W C
After Reverse shuffle	G C F I V W I G Y V M X C
After Reverse shuffle (-2)	C Y B E R S E C U R I T Y

Final Decryption Output : CYBERSECURITY

4.3.5 Test Case 5

Uppercase and Lowercase Handling

Plaintext KpOli

Key Used 24

Character Type Uppercase + Lowercase

Mapping A–Z = 0–25, a–z = 0–25

Encryption

Round 1 – Key Digit: 2

Stage	Output
After Shift	M r Q n k
After Shuffle	k n Q r M

Round 2 – Key Digit: 4

Stage	Output
After Shift	o r U v Q
After Shuffle	Q v U r o

Final Ciphertext: QvUro

Decryption

Round 1 - Key Digit: 4

Stage	Output
Before Reverse Shuffle	Q v U r o
After Reverse Shuffle	o r U v Q
After Reverse Shift (-4)	k n Q r M

Round 2 – Key Digit 2

Stage	Output
Before Reverse Shuffle	k n Q r M
After Reverse Shuffle	M r Q n k
After Reverse Shift (-4)	K p O l i

Final Decryption Output: KpOli

5. Critical Evaluation of the New Cryptographic Algorithm

The evaluation process in this section examines the strong points and weak points of the Simple Hybrid Shift-Shuffle Cipher (SHSSC) and its practical uses.

5.1 Strengths

- **Improved Security over Classical Ciphers:**

SHSSC protects messages through multiple encryption stages which include dynamic shift operations and transposition techniques to create a secure encryption system.

- **Resistance to Frequency Analysis:**

The process of character shuffling breaks down statistical patterns which classical cryptanalysis methods use for their analysis.

- **Dynamic Key Usage:**

Users must enter multi-digit numeric keys which change the encryption system through an evolving process that makes it more complex and unpredictable

- **Simplicity and Educational Value:**

The algorithm is easy to understand and implement, making it ideal for academic learning and demonstrations.

- **Deterministic and Reversible:**

Decryption reliably recovers plaintext when the correct key is used.

5.2 Limitations

Alphabet-Only Support: Only capital letters (A–Z) are supported by the current implementation; numbers and symbols are not.

Unsuitable for Applications with High Security: SHSSC does not adhere to contemporary cryptographic standards like AES, despite being more powerful than classical ciphers.

Key Exposure Risk: When a key is compromised, confidentiality is completely lost, just like with other symmetric ciphers.

Absence of Formal Cryptanalysis: Neither comprehensive mathematical security proofs nor practical cryptographic validation have been applied to the algorithm.

6. Conclusion

The main goal of this coursework, which was to investigate, create, test, and assess a novel cryptographic algorithm, was effectively accomplished. The research began with a thorough analysis of classical cryptographic systems, especially the Caesar cipher, and found important flaws like predictable patterns and constrained key space.

The Simple Hybrid Shift-Shuffle Cipher (SHSSC) was created by combining key evolution, transposition, and dynamic shifting to solve these problems. The algorithm's accuracy, reversibility, and enhanced resistance to fundamental cryptanalysis techniques were all verified by methodical testing in a variety of scenarios.

SHSSC successfully illustrates how classical ciphers can be improved through logical and mathematical changes, even though it is not meant to replace contemporary encryption standards. The project gave important insights into testing methods, security assessment, and cryptographic design principles.

7. References

Bibliography

Holdsworth, J., n.d. *ibm*. [Online]

Available at: <https://www.ibm.com/think/topics/information-security>

Hashemi-Pour, C., 2023 dec 21. *techtarget*. [Online]

Available at: <https://www.techtarget.com/whatis/definition/Confidentiality-integrity-and-availability-CIA>

GDPR, 18 June 2025. *grcsolutions.io*. [Online]

Available at: <https://grcsolutions.io/what-is-the-cia-triad-and-why-is-it-important/>

Kosinski, M., n.d. *ibm*. [Online]

Available at: https://www.ibm.com/think/topics/cryptography?utm_source=chatgpt.com

Richards, K., mar 13 , 2024. *techtarget*. [Online]

Available at:

https://www.techtarget.com/searchsecurity/definition/cryptography?utm_source=chatgpt.com

Gupta, T., 5 Jun 2024. *scaler*. [Online]

Available at: <https://www.scaler.com/topics/computer-network/cryptography-and-network-security/>

Sidhpurwala, H., January 12, 2023. *Red hat*. [Online]

Available at: <https://www.redhat.com/en/blog/brief-history-cryptography>

Schneider, J., n.d. *ibm*. [Online]

Available at: <https://www.ibm.com/think/topics/cryptography-history>

Majumder, M., July 18, 2025. *encryptionconsulting*. [Online]

Available at: <https://www.encryptionconsulting.com/symmetric-vs-asymmetric-encryption-top-use-cases-in-2025/>

Wickramasinghe, S., n.d. *Splunk Blogs*. [Online]

Available at: https://www.splunk.com/en_us/blog/learn/caesar-cipher.html

Anon., n.d. [Online]

Available at: <https://www.geeksforgeeks.org/>

geeksforgeeks, 23 Jul, 2025. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/ethical-hacking/caesar-cipher-in-cryptography/>

threatscape, n.d. *threatscape*. [Online]

Available at: <https://www.threatscape.com/cyber-security-blog/understanding-the-cia-triad-in-cyber-security/>

Sidhpurwal, H., n.d. *redhat*. [Online]

Available at: <https://www.redhat.com/en/blog/brief-history-cryptography>

TEAM, E., 2025. *egypttoursportal*. [Online]

Available at: al.com/en-us/blog/egyptian-tombs/beni-hasan/

Gupta, T., 13 Oct 2022. *scaler*. [Online]

Available at: <https://www.scaler.com/topics/computer-network/cryptography-and-network-security/>

enthusiast, c., n.d. *cocomelonc*. [Online]

Available at: <https://cocomelonc.github.io/malware/2024/10/20/malware-cryptography-33.html>

Pyrik, B., Mar 3, 2019. *observablehq*. [Online]
Available at: <https://observablehq.com/@bryik/caesar-cipher>

8. Appendices